Routy: A Small Routing Protocol

Group 3: Ilias Bourdakos, Dimitrios Chrysos

September 16, 2026

1 Introduction

This assignment implements Routy, a link-state routing protocol in Erlang. Each router is an Erlang process with a name, a link-state counter, a history recording the latest counter seen from each router, a set of interfaces, a routing table, and a directional network map. Routers broadcast their direct neighbors through link-state messages and use Dijkstra's algorithm to calculate the best next gateway for each reachable destination.

2 Main problems and solutions

- **Directional links:** A known node may still be unreachable because links only work in one direction. This affects how link knowledge is broadcasted in the network, making some nodes unaware of new changes. This can be addressed by designing a directed cyclic topology, allowing link-state messages to travel through the network. For bidirectional direct communication, two separate directed connections can be used.
- **Choosing a useful test topology:** We found it difficult to create a directional graph that demonstrated routing, link-state flooding, cycles, failures, and unreachable nodes. For the automated test, we created a five-router graph with a directed cycle and one sink node.
- **Asynchronous testing:** Router actions happen through asynchronous messages, so state changes are not visible immediately. We used status messages and short delays in our automated test to inspect the router properties (e.g. maps, tables, and interfaces).

3 Potential improvements

- **Automatic convergence and stale-data recovery:** Routers could automatically broadcast updated link-state information and rebuild routing tables whenever their interfaces or network maps change. Periodic refreshes (broadcasts) would confirm that links announced by routers are still valid, while entries that are not refreshed could expire from the map. A persistent counter would let a restarted router continue with a higher message number, so its new announcements are not treated as old.

- **Weighted routing:** Links could have different cost values instead of every link having the same cost of one hop.
- **Delivery feedback:** Routers could return acknowledgments or error messages when a packet cannot be delivered, rather than silently dropping it.
- **More efficient interfaces and history:** The current interface list requires a linear search for operations such as:
  `intf:lookup/2, intf:ref/2, intf:name/2`,

  It could be replaced by a map from router name to {`Ref, Pid`}, such as `#{Name => {Ref, Pid}}`, and a second map from monitor reference to router name for reverse lookups. History could similarly use a map from router name to its newest counter. This would avoid repeated list searches, but the two interface maps must always be updated together and require additional memory.
- **More efficient network map:** The directional map could use an Erlang map from node name to its outgoing links, making reachability lookups and map updates more efficient than searching a list.

4 Evaluation

Our automated test in `routy_tests.erl` starts five routers: Athens, Patras, Thessaloniki, Katerini, and Larisa. The directed topology is:

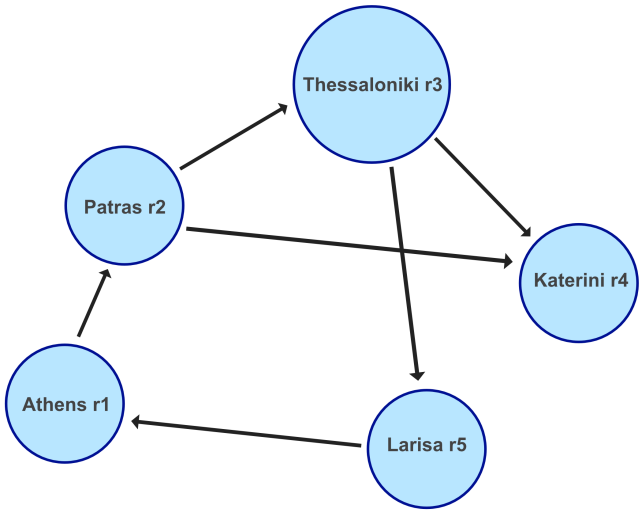


Figure 1: Directed five-router topology used by the automated test.

The links Athens → Patras → Thessaloniki → Larisa → Athens form a directed cycle. Katerini is a sink node with no outgoing interfaces.

The test broadcasts link-state messages and rebuilds the routing tables. It successfully delivers a multi-hop message from Athens to Katerini through Patras, and a message from Patras to Athens through Thessaloniki and Larisa. It also sends a packet to the unknown destination `nowhere`, which is dropped without crashing the routers.

The test then stops Larisa. Thessaloniki receives a `'DOWN'` message and removes Larisa from its interfaces. Thessaloniki's updated link-state message can initially reach only the sink node Katerini, so Athens and Patras retain stale information about Larisa.

When Larisa restarts, it is connected only through an outgoing interface from Thessaloniki and has no outgoing interface of its own. Therefore, its updated state cannot reach the other routers, even though the routing tables of the other nodes, think so. Even if its original outgoing connection to Athens were restored, its first announcement would use counter zero and be treated as old by routers that still store {`larisa, 0`}. A later announcement with a higher counter would be required to update their stale information. This demonstrates a limitation of the current protocol.

5 Conclusion

Routy demonstrates link-state routing through directional links, link-state broadcasts, Dijkstra routing tables, and forwarding packets through the appropriate gateway. The assignment also shows how counters prevent flooding loops and how monitors handle failed router processes. The failure test highlights the need for automatic convergence and stale-data recovery in a more complete routing protocol.